

Reinforcement Learning Monte Carlo Methods

Debapriyo Majumdar
Indian Statistical Institute
debapriyo@isical.ac.in

“Monte Carlo” methods

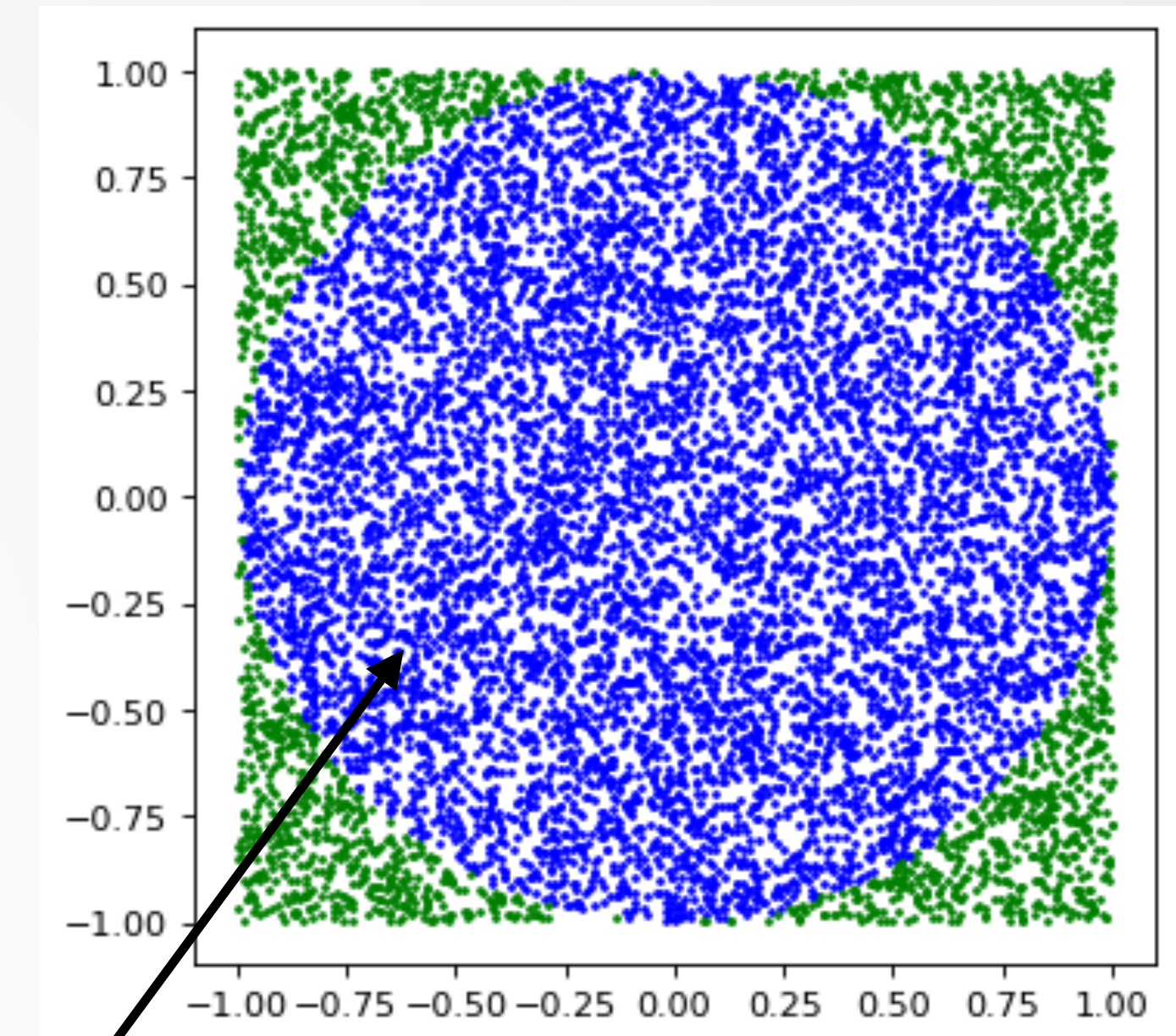
- Broadly: methods of estimating using repeated random sampling
- Law of large numbers: If $X_1, X_2, \dots, X_n$ are i.i.d., with $\mu = \mathbb{E}[X_i]$, then

$$\mathbb{P} \left[\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n X_n = \mu \right] = 1$$

- Example task: estimate the value of π , the area of the unit circle

- Approach:

- Draw coordinates of N points (x, y) uniformly at random from $(-1, 1)$
- The points are (uniform) random points in the square with area = 4
- Count the number n of points (x, y) such that $x^2 + y^2 < 1$ (inside the unit circle)
- Then, $\pi = 4n/N$
- For a large N , we will get a good estimate



```
import numpy as np

N = 1000 * 1000

X = np.random.uniform(-1, 1, N)
Y = np.random.uniform(-1, 1, N)

Z = X**2 + Y**2
count = sum(Z < 1)
print(4 * count/N)
```

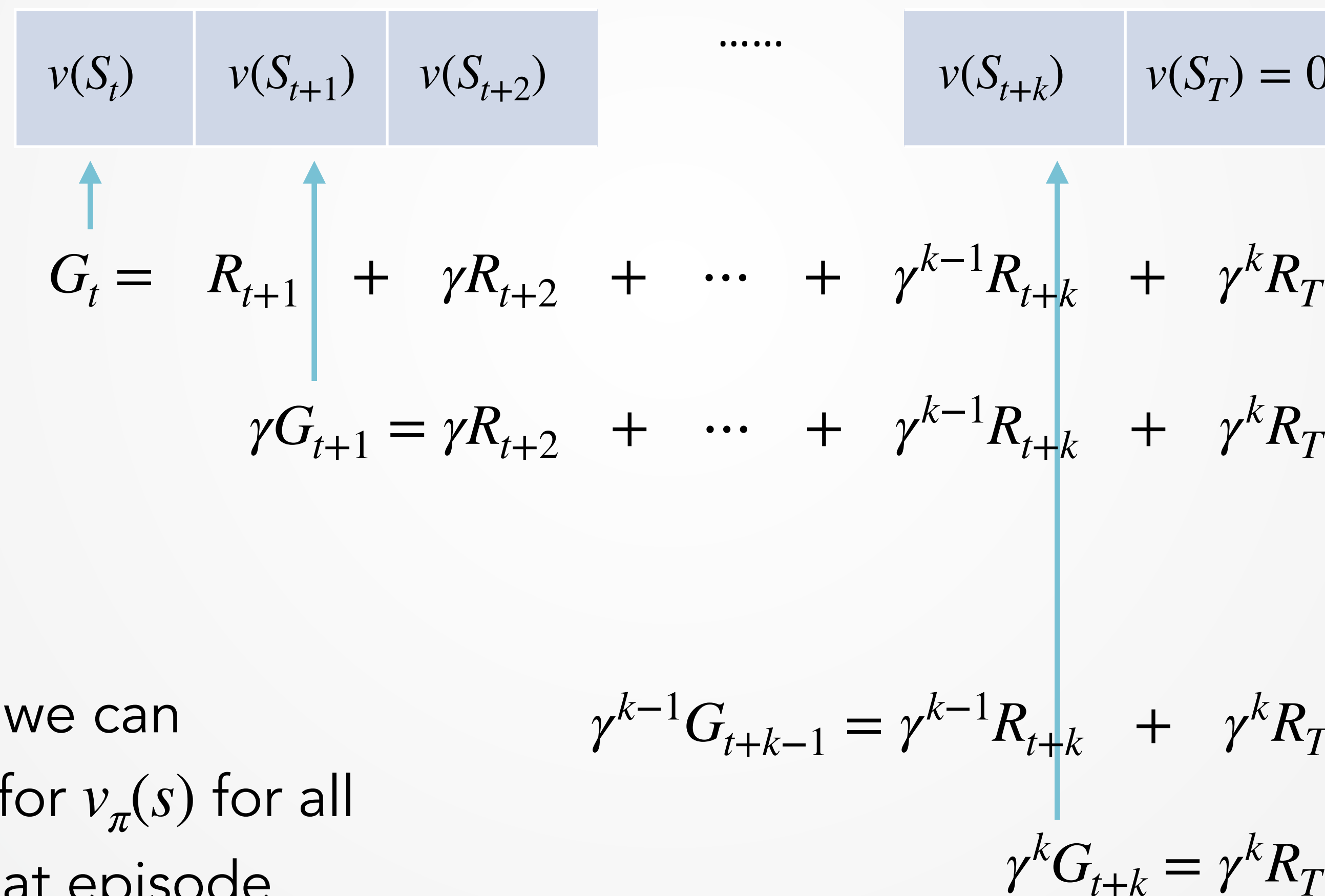
Monte Carlo method for policy evaluation

- Goal of policy evaluation: estimate state values $v_{\pi}(s)$ of a policy π
- We do not need to know the dynamics p of the environment
- By definition: $v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s]$
- Monte Carlo approach:
 - Estimate the expected return from many samples of G_t with $S_t = s$
 - Repeated sampling of episodes
 - Compute sample returns from each state visited
 - Update average return for the states as new sample returns are obtained
 - Recall: need to only maintain number of samples and current average to update average

Returns from an episode

An episode from any state S_t

$$S_t \longrightarrow S_{t+1} \longrightarrow S_{t+2} \longrightarrow \dots \longrightarrow S_{t+k} \longrightarrow S_T$$



From one episode, we can obtain one sample for $v_\pi(s)$ for all states s visited in that episode

Policy evaluation algorithm: Monte Carlo

- Goal: evaluate a policy π
- Algorithm:

Initialize the estimate $V(s) \in \mathbb{R}$ arbitrarily for all

The number of samples $n(s) := 0$ for all $s \in \mathcal{S}$

Loop forever

Generate an episode following π , as: $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

for each $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Update $V(S_t) := \frac{1}{n(S_t) + 1} (n(S_t)V(S_t) + G)$

Update $n(S_t) := n(S_t) + 1$

First visit vs every visit

- In an episode, what if a state occurs more than once?
- $S_t = s \longrightarrow S_{t+1} \longrightarrow S_{t+2} \longrightarrow \dots \longrightarrow S_{t+l} = s \longrightarrow \dots \longrightarrow S_{t+k} \longrightarrow S_T$
- First visit Monte Carlo: we only consider the first visit to a state s in an episode to collect a sample of $v_\pi(s)$
- Every visit Monte Carlo: we consider all visits (the algorithm we have seen)

First visit Monte Carlo

- Goal: evaluate a policy π
- Algorithm:

Initialize the estimate $V(s) \in \mathbb{R}$ arbitrarily for all

The number of samples $n(s) := 0$ for all $s \in \mathcal{S}$

Loop forever

Generate an episode following π , as: $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

for each $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Unless $S_t \in \{S_0, S_1, \dots, S_{t-1}\}$

Update $V(S_t) := \frac{1}{n(S_t) + 1} (n(S_t)V(S_t) + G)$

Update $n(S_t) := n(S_t) + 1$

First visit vs every visit

- In an episode, what if a state occurs more than once?
- $S_t = s \longrightarrow S_{t+1} \longrightarrow S_{t+2} \longrightarrow \dots \longrightarrow S_{t+l} = s \longrightarrow \dots \longrightarrow S_{t+k} \longrightarrow S_T$
- First visit Monte Carlo: we only consider the first visit to a state s in an episode to collect a sample of $v_\pi(s)$
- Every visit Monte Carlo: we consider all visits (the algorithm we have seen)
- Both algorithms make the estimate $V(s)$ converge to $v_\pi(s)$ for every state s as the number of samples for each state is large enough ($\rightarrow \infty$)
- Every visit (1996) is more modern and converges faster

Estimating action values

- Instead of estimating $v_\pi(s)$, we estimate $q_\pi(s, a)$ for state-action pairs
- An episode: $S_t \xrightarrow{A_t} S_{t+1} \xrightarrow{A_{t+1}} S_{t+2} \xrightarrow{A_{t+2}} \dots \xrightarrow{A_{t+k-1}} S_{t+k} \xrightarrow{A_{t+k}} S_T$
- Similar to the state-value estimate process, estimate by sampling values of $q(S_t, A_t)$
- First visit and every visit variants as before
- Key difference between dynamic programming (DP) and Monte Carlo (MC)
 - In DP, the assumption is we know the dynamics (p), so we can use the following equation
$$v_\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]$$
 - Given a state s , for every action a , we know the probability of the rewards
 - From a state value we can look ahead and choose the best action (for greedy improvement)
 - In MC, we don't know the dynamics p
 - To know which action a is best from a state s , we need to actually sample (s, a) pairs and estimate action values explicitly

Monte Carlo policy control

Initialize state-action probability $\pi(s) \in \mathcal{A}$ arbitrarily

Initialize state action values $Q(s, a) \in \mathbb{R}$ arbitrarily for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize number of samples $n(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$

Loop forever

Choose starting points A_0 and S_0

Generate episode following $\pi : S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

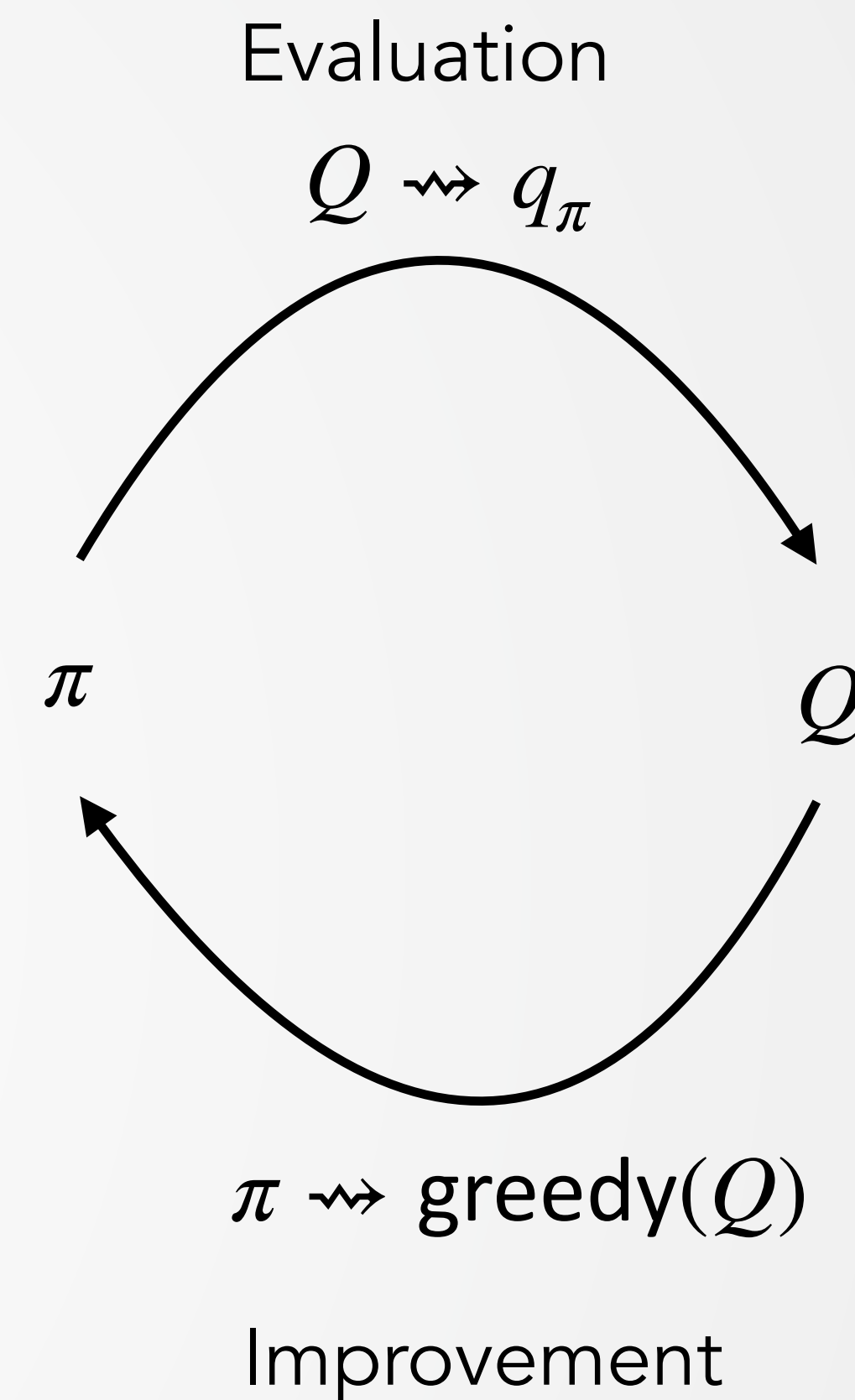
For each step $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Update $Q(S_t, A_t) := \frac{1}{n(S_t, A_t) + 1} (n(S_t, A_t)Q(S_t, A_t) + G)$

Update $n(S_t, A_t) := n(S_t, A_t) + 1$

Update policy $\pi(S_t) := \arg \max_a Q(S_t, a)$



Monte Carlo policy control

Initialize state-action probability $\pi(s) \in \mathcal{A}$ arbitrarily

Initialize state action values $Q(s, a) \in \mathbb{R}$ arbitrarily for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize number of samples $n(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$

Loop forever

Choose starting points A_0 and S_0 randomly (exploring start)

Generate episode following $\pi : S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

For each step $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Update $Q(S_t, A_t) := \frac{1}{n(S_t, A_t) + 1} (n(S_t, A_t)Q(S_t, A_t) + G)$

Update $n(S_t, A_t) := n(S_t, A_t) + 1$

Update policy $\pi(S_t) := \arg \max_a Q(S_t, a)$

Possible issue:

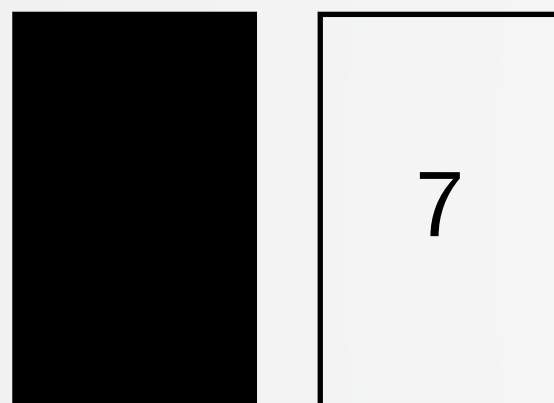
If the policy π is deterministic, some state-action pairs may never be sampled

One solution: exploring start

Choose the starting pair (S_0, A_0) randomly so that all possible state-actions pairs have probability > 0 , and are eventually encountered

Blackjack

Dealer



Counting: all face cards = 10
Ace = 1 or 11

Goal: get as close as possible,
but not exceeding 21

Player



Natural: win

Blackjack

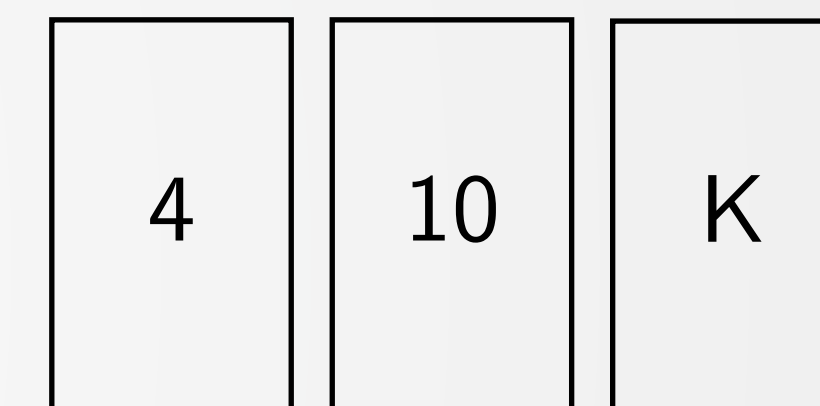
Dealer

Player



Counting: all face cards = 10
Ace = 1 or 11

Goal: get as close as possible,
but not exceeding 21

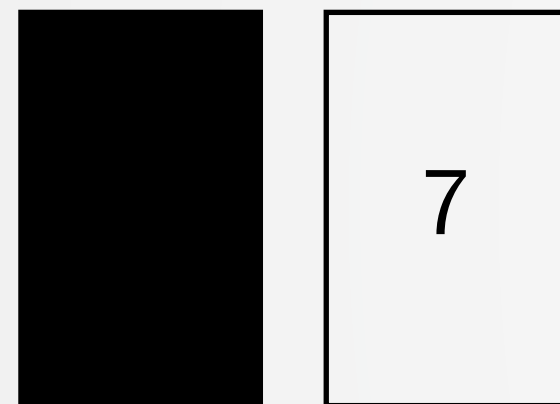


- Actions (deterministic):
 - **Hit** (draw more cards, one at a time)
 - **Stick** (stop) at anything ≥ 17
- Outcomes:
 - **Go bust** (> 21) $\rightarrow$ lose
 - Both stick at $\leq 21 \rightarrow$ one with higher point wins

- Possible actions (player chooses):
 - **Hit** (request more cards, one at a time)
 - **Stick** (stop)
- Outcomes:
 - **Go bust** (> 21) $\rightarrow$ lose
 - **Stick** $\leq 21 \rightarrow$ Dealer's turn

Blackjack

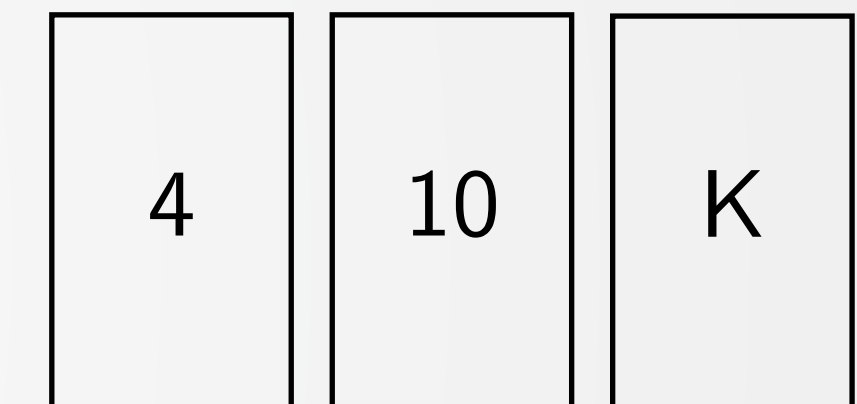
Dealer



Counting: all face cards = 10
Ace = 1 or 11

Goal: get as close as possible,
but not exceeding 21

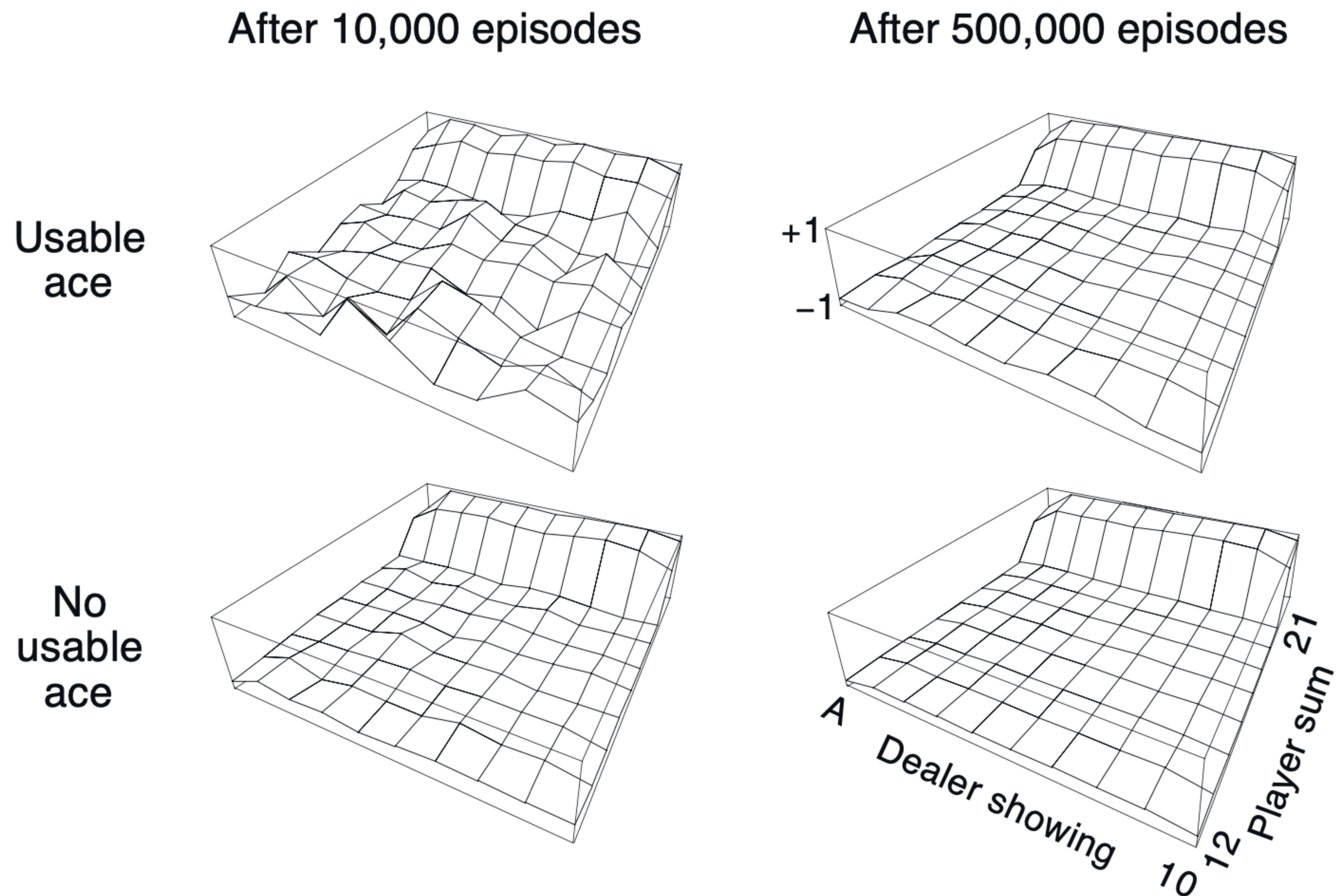
Player



- Episodic finite MDP (no discounting)
- Inputs for strategy: player's cards and dealer's open card
 - Note: if the sum is ≤ 11 the player should always **hit** (obvious strategy, no learning required)
 - Usable Ace: an Ace that can be counted as 11 without going bust
 - Scenarios: player's sum (12 - 21), dealer's open card (Ace - 10), whether player has an usable Ace
 - Number of states: $10 \times 10 \times 2 = 200$

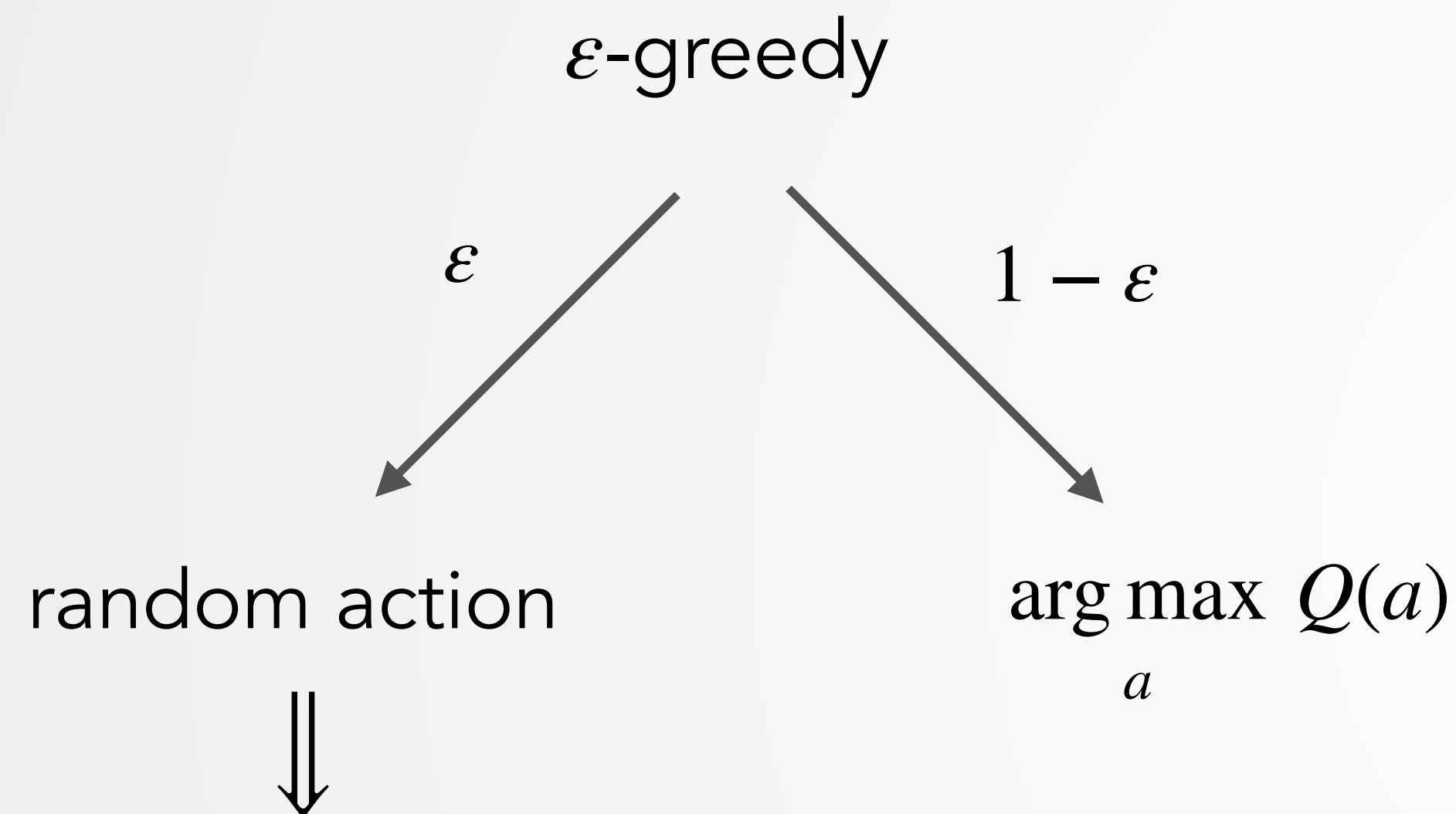
State Value Function for a Policy which sticks only at 20 or 21

15



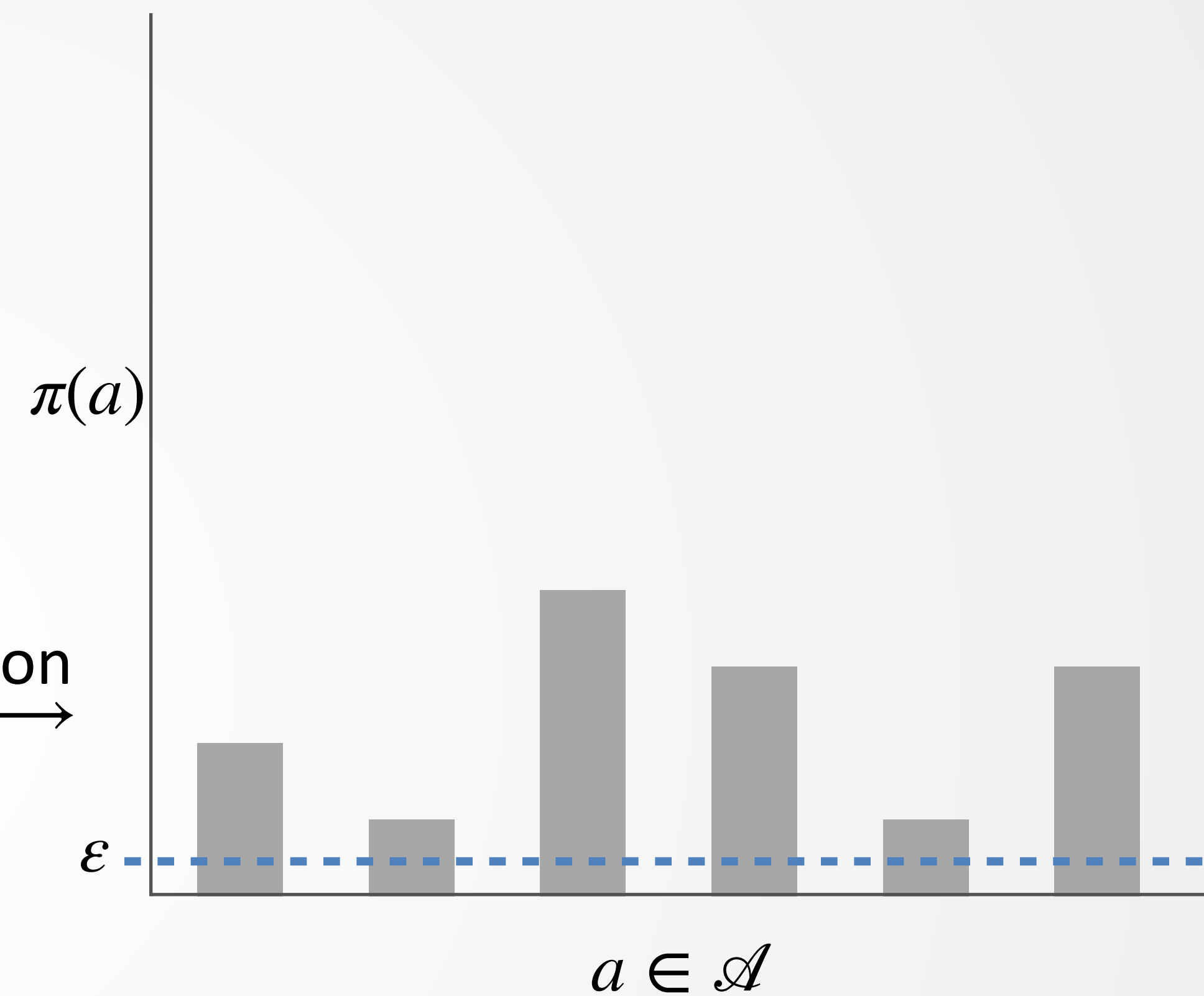
Exploration with ε -soft policies

Recall ε -greedy policy



If number of actions is $|\mathcal{A}|$, then all actions have probability $\frac{\varepsilon}{|\mathcal{A}|}$, except $\arg \max_a Q(a)$, which has a higher probability

generalization



ε -soft policy

All actions have probability $\geq \frac{\varepsilon}{|\mathcal{A}|}$

Exploring start to ϵ -soft: discussion

- Exploring start is unreasonable in some scenarios
 - Cannot ensure that the agent can start at any (state-action) pair
 - For example, self-driving car
- Need to keep exploring instead of only rely on the start
 - The ϵ -soft approach helps in this case, explore always

Monte Carlo policy control (ϵ -soft)

Initialize $\pi(s) \in \mathcal{A}$ arbitrarily, $Q(s, a) \in \mathbb{R}$ arbitrarily for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize number of samples $n(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$

Loop forever

Choose starting points A_0 and S_0 randomly $\longleftarrow$ **Remove the exploring start condition**

Generate episode following $\pi : S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

For each step $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Update $Q(S_t, A_t) := \frac{1}{n(S_t, A_t) + 1} (n(S_t, A_t)Q(S_t, A_t) + G)$

Update $n(S_t, A_t) := n(S_t, A_t) + 1$

Update policy $\pi(S_t) := \arg \max_a Q(S_t, a)$ $\longleftarrow$ **Implement ϵ -soft update here**

Monte Carlo policy control (ϵ -soft)

Initialize $\pi(s) \in \mathcal{A}$ arbitrarily, $Q(s, a) \in \mathbb{R}$ arbitrarily for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize number of samples $n(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$

Loop forever

Generate episode following $\pi : S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0$

For each step $t = T - 1, T - 2, \dots, 0$

$G := \gamma G + R_{t+1}$

Update $Q(S_t, A_t) := \frac{1}{n(S_t, A_t) + 1} (n(S_t, A_t)Q(S_t, A_t) + G)$

Update $n(S_t, A_t) := n(S_t, A_t) + 1$

Set $A_* := \arg \max_a Q(S_t, a)$

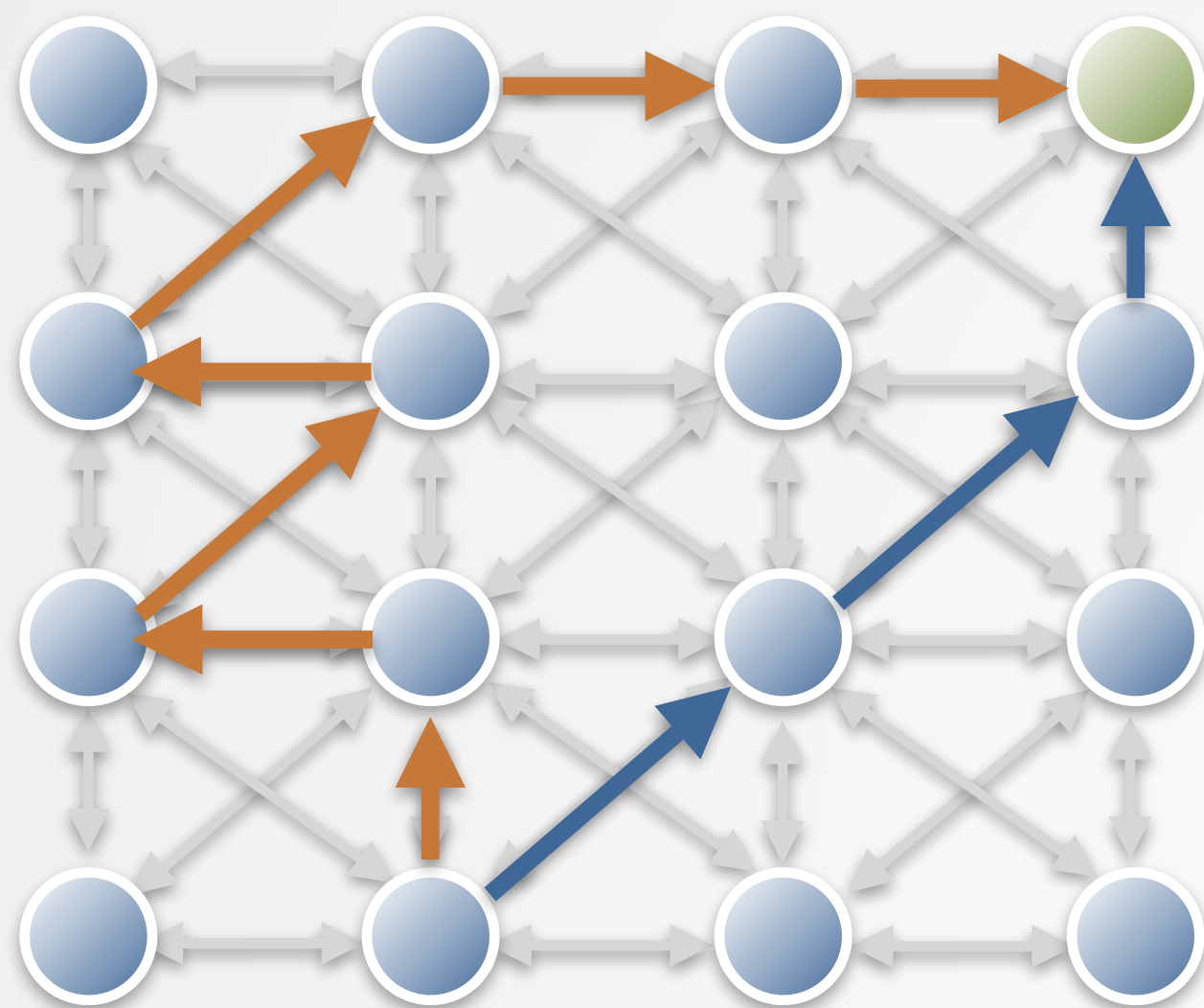
For all $a \in \mathcal{A}$

$$\pi(a | S_t) := \begin{cases} 1 - \epsilon + \epsilon / |\mathcal{A}| & \text{if } a = A_* \\ \epsilon / |\mathcal{A}| & \text{otherwise} \end{cases}$$

Exploring start to ϵ -soft: discussion

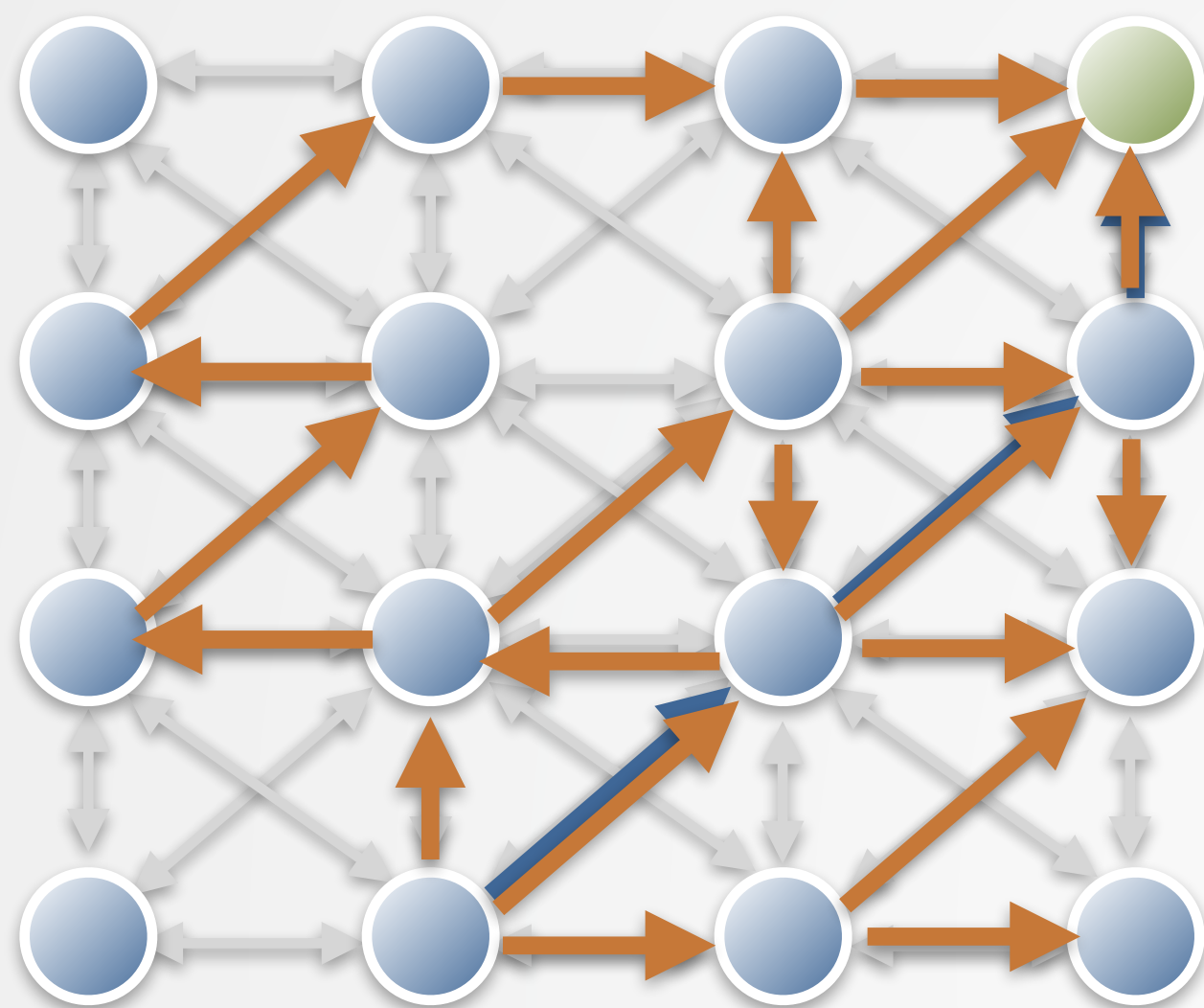
- Exploring start is unreasonable in some scenarios
 - Cannot ensure that the agent can start at any (state-action) pair
 - For example, self-driving car
- Need to keep exploring instead of only rely on the start
 - The ϵ -soft approach helps in this case, explore always
- Issues
 - Always suboptimal, because keeps exploring always
- Dilemma:
 - Want to learn action values on conditional subsequent optimal behavior
 - But need to behave non-optimally to explore all actions and learn

On-policy and off-policy learning



- So far we have considered **on-policy** learning
 - Improve and evaluate the policy used to generate the data
 - More specifically, learn the value functions
- **Off-policy learning**
 - Improve and evaluate a policy that is different from the one used to generate the data
 - **Target policy**: the policy π we want to learn
 - **Behavior policy**: the policy we follow to select the actions
 - Usually more exploratory
 - *Important*: the behavior policy must cover the target policy
 - Must *also* take the actions which the target policy takes

On-policy and off-policy learning

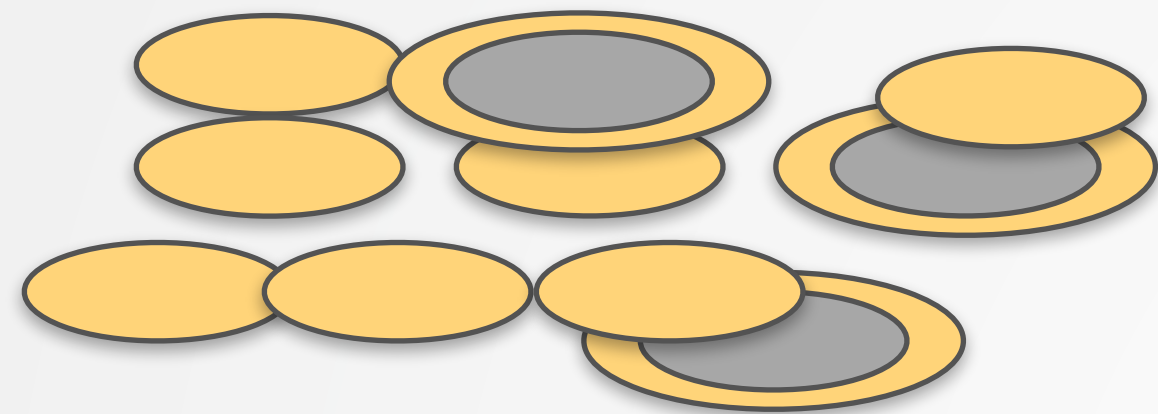


If $\pi(a | s) > 0$ then $b(a | s) > 0$

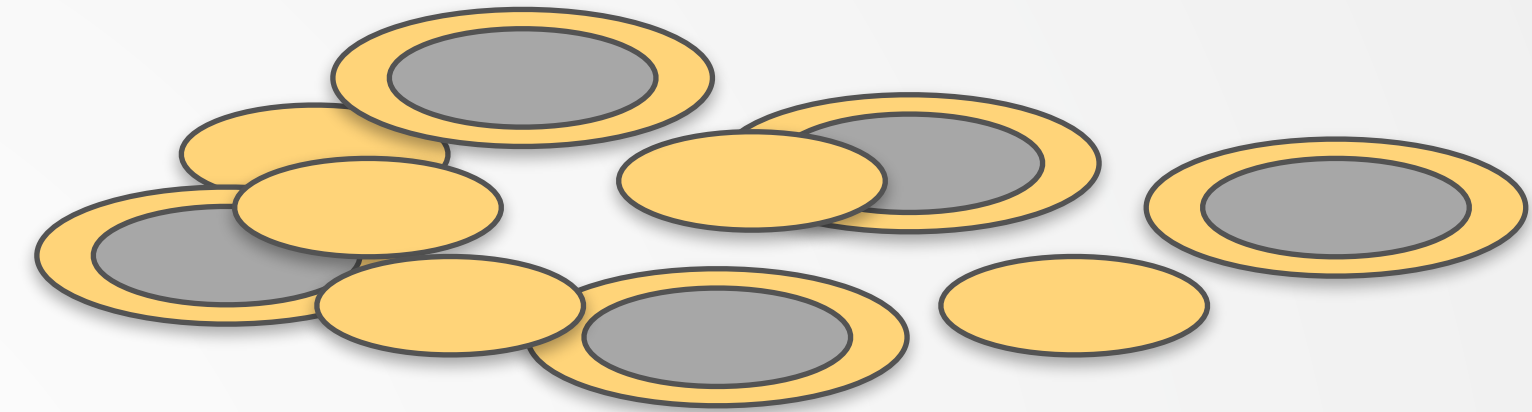
- So far we have considered **on-policy** learning
 - Improve and evaluate the policy used to generate the data
 - More specifically, learn the value functions
- **Off-policy learning**
 - Improve and evaluate a policy that is different from the one used to generate the data
 - **Target policy**: the policy π we want to learn
 - **Behavior policy**: the policy b we follow to select the actions
 - Usually more exploratory
 - *Important*: the behavior policy must cover the target policy
 - Must *also* take the actions which the target policy takes

Estimating one distribution by sampling from another 23

70% five rupee coins and 30% ten rupee coins



50% five rupee coins and 50% ten rupee coins



X = value of a coin drawn from the pile (a random variable)

$$\pi(5) = 0.7, \pi(10) = 0.3$$

$$\mathbb{E}_{\pi}(X) = 5 \times 0.7 + 10 \times 0.3 = 6.5$$

$$b(5) = 0.5, b(10) = 0.5$$

$$\mathbb{E}_b(X) = 5 \times 0.5 + 10 \times 0.5 = 7.5$$

Importance sampling: we want to estimate $\mathbb{E}_{\pi}(X)$ sampling from distribution b

Importance sampling

- By definition, we have:

$$\begin{aligned}\mathbb{E}_\pi[X] &= \sum_{x \in X} x \pi(x) \\ &= \sum_{x \in X} x \pi(x) \frac{b(x)}{b(x)} \text{ (here, we need that whenever } \pi(x) > 0, \text{ we have } b(x) > 0) \\ &= \sum_{x \in X} x \frac{\pi(x)}{b(x)} b(x) \\ &= \sum_{x \in X} x \rho(x) b(x), \text{ where } \rho(x) = \frac{\pi(x)}{b(x)} \text{ is called the importance sampling ratio} \\ &= \mathbb{E}_b [X \rho(X)], \text{ expectation of the random variable } X \rho(X) \text{ under distribution } b \\ &\approx \frac{1}{n} \sum_{i=1}^n x_i \rho(x_i), \text{ where } x_i \sim b\end{aligned}$$

- We can estimate it by sampling x_i from b , but taking average of $x_i \rho(x_i)$ instead of x_i

Importance Sampling Ratio

- Target policy π , behavior policy b
- We want to estimate $v_\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$
- On-policy: we follow policy π , collect samples of G_t given $S_t = s$
 - Then we estimate $v_\pi(s)$ by averaging samples of G_t
- Off-policy: we need to compute the average ρG_t
$$v_\pi(s) = \mathbb{E}_b [\rho G_t | S_t = s]$$
- We need to compute ρ for each sample

Importance Sampling Ratio

- Consider a particular episode under policy b starting at step t till the terminal step T
- Trajectory: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots, S_{T-1}, A_{T-1}, R_T, S_T$
- The importance sampling ratio ρ is
$$\rho = \frac{P [\text{trajectory under } \pi]}{P [\text{trajectory under } b]}$$
- How is a trajectory determined when the agent follows a policy b ?
- Agent takes action (following policy b), goes to a state and gets a reward (according to the dynamics p of the environment)

Importance Sampling Ratio

- Probability of a trajectory under policy b is $P \left[S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots, S_{T-1}, A_{T-1}, R_T, S_T \mid S_t, A_{t:T-1} \right]$

where the actions $A_t, \dots, A_{T-1}$ are taken according to b

- We can write this as

$$b(A_t | S_t)P \left[S_{t+1}, R_{t+1} \mid S_t, A_t \right] b(A_{t+1} | S_{t+1})P \left[S_{t+2}, R_{t+2} \mid S_{t+1}, A_{t+1} \right] \cdots b(A_{T-1} | S_{T-1})P \left[S_T, R_T \mid S_{T-1}, A_{T-1} \right]$$

$$= \prod_{k=t}^{T-1} b(A_k | S_k)P[S_{k+1}, R_{k+1} | S_k, A_k]$$

- So, $\rho_{t:T-1} = \frac{P \left[\text{trajectory under } \pi \right]}{P \left[\text{trajectory under } b \right]} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)P[S_{k+1}, R_{k+1} | S_k, A_k]}{b(A_k | S_k)P[S_{k+1}, R_{k+1} | S_k, A_k]} = \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$

- The dynamics (P) of the MDP are not known, but that's okay
- Since we know π and b , we can compute ρ for any trajectory
- In fact, we can compute it by backtracking

Off-policy Monte Carlo evaluation

- Goal: evaluate a policy π , but follow policy b
- Algorithm:

Initialize the estimate $Q(s, a) \in \mathbb{R}$ arbitrarily and the cumulative weight $c(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$

Loop forever

Generate an episode following b , as: $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0, W := 1$

for each $t = T - 1, T - 2, \dots, 0$ while $W \neq 0$

$$G := \gamma G + R_{t+1}$$

$$c(S_t, A_t) := c(S_t, A_t) + W$$

$$\text{Update } Q(S_t, A_t) := Q(S_t, A_t) + \frac{W}{c(S_t, A_t)} [G - Q(S_t, A_t)]$$

$$\text{Update } W := W \frac{\pi(A_t | S_t)}{b(A_t | S_t)}$$

Off-policy Monte Carlo Control

- Goal: estimate $\pi \sim \pi^*$, off-policy
- Algorithm:

Initialize the estimate $Q(s, a) \in \mathbb{R}$ arbitrarily and the cumulative weight $c(s, a) := 0$ for all $s \in \mathcal{S}, a \in \mathcal{A}$ and $\pi(s) = \operatorname{argmax}_a Q(s, a)$

Loop forever (for each episode)

Take b to be any “soft” policy

Generate an episode following b , as: $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G := 0, W := 1$

for each $t = T - 1, T - 2, \dots, 0$ while $W \neq 0$

$$G := \gamma G + R_{t+1}; \quad c(S_t, A_t) := c(S_t, A_t) + W; \quad Q(S_t, A_t) := Q(S_t, A_t) + \frac{W}{c(S_t, A_t)} [G - Q(S_t, A_t)]$$

Set $\pi(S_t) := \operatorname{argmax}_a Q(S_t, a)$

If $A_t \neq \pi(S_t)$, exit inner loop (proceed to next episode)

Update $W := W \frac{1}{b(A_t | S_t)}$

References

- Sutton, R.S. and Barto, A.G., 2018. Reinforcement learning: An introduction. MIT press.
<http://incompleteideas.net/book/the-book.html> (primary reference), Chapter 5